

# Note per Distributed Random Sampling

Andrea Clementi

March 2026

## 1 Introduzione

### 1.1 Sampling da un Insieme di Dati: Algoritmi Centralizzati

Il primo task che studieremo è quello di generare e mantenere un *Sampling* di dimensione *fissa*, senza rimpiazzo, da un *data set*  $A$  di  $n$  elementi. In particolare, mostreremo i seguenti risultati:

1. **Uniform Random Sample da un Array di Dati.** Dato un insieme  $A$  di  $n$  elementi, memorizzati in un array  $\vec{A} = \vec{A}[1 : n]$ , l'algoritmo *Fisher-Yates Shuffle* genera, in modo iterativo, una permutazione u.a.r.  $\vec{A}_\pi$  di  $\vec{A}$ . Allora, se si definisce  $\mathcal{S}$  come i primi  $s$  elementi di  $\vec{A}_\pi$ , mostreremo che  $\mathcal{S}$  soddisfa le seguenti proprietà:
  - a). Inteso come sottoinsieme non ordinato,  $\mathcal{S}$  è distribuito uniformemente sullo spazio (che denoteremo con  $\binom{n}{s}$ ) di tutti i sottoinsiemi di size  $s$  di  $A$ ;
  - b). Inoltre,  $\vec{\mathcal{S}} = \vec{\mathcal{S}}[1 : s]$ , inteso come insieme ordinato, è ordinato in modo u.a.r. rispetto a tutte i possibili  $s!$  ordinamenti.

Questa parte sarà trattata nella Section 2.

2. **Uniform Random Sample da uno Stream di Dati.** Consideriamo un modello in cui i dati di input arrivano ad un server  $C$  in modalità *streaming*  $a_1, a_2, \dots, a_k, \dots$ . Il task che  $C$  deve eseguire consiste nel generare e mantenere *on-line* un sampling  $\mathcal{S}(k)$  di size  $s$  senza rimpiazzo che abbia le seguenti proprietà:
  - a1). Alla fine del round  $k$ , l'algoritmo ha letto in modalità on-line i primi  $k$  elementi, ed il suo output, inteso come sottoinsieme non ordinato,  $\mathcal{S}(k)$  deve essere distribuito uniformemente sullo spazio di tutti i sottoinsiemi di size  $s$  di  $\vec{A}[1, k]$ ;
  - b1). L'algoritmo deve mantenere una permutazione random di  $\mathcal{S}(k)$ .

Questa parte verrà trattata nella Section 3

### 1.2 Distributed Sampling

**Il Merge di Random Sampling Distribuiti.** Consideriamo ora un modello di calcolo distribuito a stella formato da un server centrale  $C$  e da  $m \geq 2$  nodi periferici  $P_1, \dots, P_m$ .

Il nodo centrale  $C$  deve calcolare e mantenere un sampling  $\mathcal{S}$  del data set  $A$  globale che è partizionato in  $m$  sottoinsiemi disgiunti  $A_1, \dots, A_m$ , ognuno memorizzato e gestito privatamente da un nodo periferico: il nodo centrale non ha accesso diretto ai dati dei nodi periferici.

In questo setting distribuito, il task consiste nel generare localmente dei random samplings  $\mathcal{S}_1, \dots, \mathcal{S}_m$  da parte dei singoli nodi periferici e fare in modo che  $C$  possa generare  $\mathcal{S}$  comunicando

con i nodi periferici. Lo scopo, oltre alla correttezza del protocollo, è quello di ottimizzare la message complexity.

Un protocollo banale per risolvere il problema è quello in cui i nodi periferici inviano al nodo centrale i rispettivi sampling insieme alle rispettive size  $n_i$  dei data set locali  $A_i$ . Il nodo centrale può a quel punto generare un unico random sampling facendo un'operazione di merge interna. Questo approccio però ha message complexity  $\tilde{\Theta}(m \cdot s)$ .

Faremo vedere un protocollo ottimale per generare  $\mathcal{S}$  applicando il Principle of Deferred Decisions che ha message complexity  $\tilde{\Theta}(m + s)$ .

Questo argomento verrà trattato nella Section 4

## 2 Permutazioni Random e Sampling

### 2.1 Permutazioni Random: L'Algoritmo di Fisher-Yates

Dato un insieme  $A$  di  $n$  elementi, memorizzati in un array  $\vec{A} = \vec{A}[1 : n]$ , l'algoritmo *Fisher-Yates Shuffle* (in breve FYA) genera, in modo iterativo, una permutazione u.a.r.  $\vec{A}_\pi$  di  $A$ .

#### **Algoritmo FYA**

**Input:**  $\vec{A}[1 : n]$  di  $n$  interi distinti

**Output:** una permutazione u.a.r.  $\vec{A}_\pi$  di  $\vec{A}$

**Set**  $\vec{A}_\pi[1 : n] := A[1 : n]$

**For**  $i = 1$  **To**  $n$  **Do**

- **Choose**  $j \in_u [i]$ ;
- **Swap**  $\vec{A}_\pi[i]$  with  $\vec{A}_\pi[j]$ .

**Return**  $\vec{A}_\pi[1 : n]$

Il fatto che il vettore finale  $\vec{A}_\pi[1 : n]$  sia ordinato u.a.r. è una conseguenza diretta del seguente

**Lemma 2.1.** *Alla fine di ogni round  $i = 1, \dots, n$ , il s.vettore prefisso  $\vec{A}_\pi[1 : i]$  è una permutazione random dei primi  $i$  elementi del vettore iniziale  $A[1 : n]$ .*

*Proof.* Il lemma si prova mediante induzione su  $i$ . Fissiamo una qualunque s.permutazione  $\langle a_1, \dots, a_i \rangle$  del s.vettore (iniziale)  $A[1 : i]$ , e calcoliamo, alla fine del round  $i \geq 1$ , la probabilità dell'evento  $\mathcal{E}(i)$ , così definito:

$$\mathcal{E}(i) = \text{"}\vec{A}_\pi[1 : i] = \langle a_1, \dots, a_i \rangle\text{"}$$

Per  $i = 1$ , l'asserto  $\mathbb{P}(\mathcal{E}(1)) = 1/1! = 1$ , dopo il primo round è banalmente vero. Possiamo quindi assumere che, per  $i \geq 2$ , alla fine del round  $i - 1$ ,  $\mathbb{P}(\mathcal{E}(i - 1)) = 1/(i - 1)!$  e dimostrare il lemma per il round  $i$ . Ora, solo due casi (i.e. eventi) (mutuamente esclusivi) possono avvenire, dipendentemente dalla scelta random  $j = J(i)$  nell'ultimo round  $i$ .

*Caso a).*  $J(i) = i$ . In tal caso, l'unica configurazione favorevole all'evento  $\mathcal{E}(i)$  era che sin dall'inizio " $\vec{A}[i] = a_i$ ". Siccome, dall'algoritmo FYA,  $\mathbb{P}(J(i) = i) = 1/i$ , dalla mutua-indipendenza delle scelte random nei round, e dalla ipotesi induttiva, abbiamo che

$$\mathbb{P}(\mathcal{E}(i)) = \frac{1}{(i - 1)!} \cdot \frac{1}{i}$$

Caso b).  $J(i) = k$  per un qualche  $k < i$ . Sia  $a_i = x$ . In questa situazione, l'unica configurazione favorevole all'evento  $\mathcal{E}(i)$  era quella in cui alla fine del round  $(i - 1)$  avevamo il s.vettore

$$\vec{\mathcal{A}}_\pi[1, (i - 1)] = \langle a_1, \dots, a_k = x, \dots, a_{(i-1)} \rangle$$

Ma allora anche in questo secondo caso la configurazione è unica e quindi si può ripetere lo stesso argomento del caso (a). □

## 2.2 Sampling Uniformi di Dimensione Fissata

Dato un insieme  $A$  di  $n$  elementi, memorizzati in un array  $\vec{\mathcal{A}}[1 : n]$ , l'algoritmo *Fisher-Yates Shuffle* FYA genera una permutazione u.a.r.  $\vec{\mathcal{A}}_\pi[1 : n]$  di  $\vec{\mathcal{A}}[1 : n]$ .

**Problema P1.** *Come possiamo utilizzare FYA per generare un sampling di size  $s \leq n$  (tipicamente  $s \ll n$ ) uniforme dell'insieme degli elementi in  $\vec{\mathcal{A}}[1 : n]$ ?*

La soluzione è molto semplice ed è descritta qui sotto (denotiamo con  $A$  l'insieme *non ordinato* di elementi di  $\vec{\mathcal{A}}[1 : n]$ ).

**Algoritmo** FY-SAMPLE

**Input:** Array  $\vec{\mathcal{A}}[1 : n]$  di numeri distinti;  $s \in [n]$

**Output:** un sampling  $\mathcal{S} \subseteq A$

**Set**  $\mathcal{S} := \emptyset$

**Execute** FYA[ $\vec{\mathcal{A}}[1 : n]$ ]

**Return**  $\mathcal{S} := \vec{\mathcal{A}}_\pi[1 : s]$ .

In breve, si esegue l'algoritmo FYA e si sceglie  $\mathcal{S}$  come i primi  $s$  elementi di  $\vec{\mathcal{A}}_\pi[1 : n]$ .

**Lemma 2.2.** *Sia  $\vec{\mathcal{A}}[1 : n]$  un vettore di  $n$  numeri distinti e sia  $s \in [n]$ . La distribuzione del random sample  $\mathcal{S}$  generato da FY-SAMPLE su input  $(A[1 : n]; s)$ , soddisfa le seguenti due proprietà:*

**a).** *Inteso come sottoinsieme non ordinato,  $\mathcal{S}$  è distribuito uniformemente sullo spazio  $\binom{n}{s}$  di tutti i sottoinsiemi di size  $s$  di  $A$ ;*

**b).**  *$\vec{\mathcal{S}}[1 : s]$ , inteso come vettore, è ordinato in modo u.a.r. rispetto a tutte i possibili  $s!$  ordinamenti.*

*Proof.* Per il Claim (a), osserviamo che, fissato un qualunque sottoinsieme  $S \subseteq A$  di size  $s$ , siccome, FYA genera una permutazione u.a.r.  $\vec{\mathcal{A}}_\pi[1 : n]$  (Lemma 2.1), abbiamo che

$$\mathbb{P}(\mathcal{S} := \vec{\mathcal{A}}_\pi[1 : s] = S) = \frac{s!(n-s)!}{n!} = \frac{1}{\binom{n}{s}}$$

Il Claim (b) è (anch'esso) una conseguenza diretta del Lemma 2.1: si noti che, per ottenere sia il Claim (a) che (b), il lemma va applicato alla fine, i.e. quando  $i = n$ . □

## 3 Sampling da uno Stream di Dati

Nella section precedente abbiamo considerato un framework centralizzato in cui gli algoritmi hanno accesso ad una memoria centrale in cui tutto il vettore dei dati è presente sin dall'inizio.

In questa section, invece, consideriamo il classico modello *Streaming* in cui l'algoritmo riceve in input una sequenza (stream) di dati (potenzialmente infinita)

$$\text{STREAMING INPUT: } a_1, \dots, a_k, \dots, \quad (1)$$

Più precisamente, l'algoritmo al round  $k \geq 1$  riceve l'elemento  $a_k$  e deve elaborare e prendere decisioni sulla base di quello che ha letto (ed eventualmente memorizzato) della sottosequenza  $a_1, \dots, a_k$ . Il modello è *one-passing*: non è possibile recuperare un elemento passato dello streaming a meno che non sia stato memorizzato nella memoria interna del processore. Tipicamente, lo scopo è quello di utilizzare meno spazio di memoria possibile per rispondere a certe fissate queries sullo stream nel più breve tempo (*query time*) possibile.

### 3.1 Reservoir Sampling su Streaming

Il task sul suddetto modello online che analizziamo in questa section è il seguente:

**Problema P2.** Su Streaming Input 1, generare e mantenere, ad ogni round  $k \geq 1$ , un sampling, di size  $s$ , distribuito u.a.r.,  $\mathcal{S}(s, k)$  dell'insieme in input di elementi  $a_1, \dots, a_k$ .

L'obiettivo quindi consiste nel mantenere, ad ogni round  $k \geq s$ , un sottoinsieme random  $\mathcal{S}(s, k)$  di size esattamente  $s$  (tipicamente  $s \ll n$ ), tale che

$$\forall S' \in \binom{[k]}{s}, \quad \mathbb{P}(\mathcal{S}(s, k) = S') = \frac{1}{\binom{k}{s}} = \frac{s!(k-s)!}{k!} \quad (2)$$

Ecco una descrizione dell'algoritmo RSA su streaming input  $a_1, \dots, a_k, \dots$ , dove al round  $k \geq 1$ , l'algoritmo riceve in input l'elemento  $a_k$ .

#### **Reservoir Sampling Algorithm** RSA

**Input:** Input Stream (1);  $s \in [n]$

**Set**  $\mathcal{S}(s, k) := \emptyset$ ;

**For Each**  $k \leq s$ , **Do**  $\mathcal{S}(s, k) := \mathcal{S}(s, k) \cup \{a_k\}$ ;

**For**  $k = s + 1, \dots$ , **Do**

**Choose**  $j \in_u [k]$ ;

**If**  $j \leq s$  **Then Replace**  $a_j$  with  $a_k$  \*\* Case (i) ( $a_j$  is removed from the Sample) \*\*

**If**  $j \geq s + 1$  **Then Discard**  $a_k$  \*\* Case (ii) \*\*

**Maintain and Return**  $\mathcal{S}(s, k)$ .

Spesso ci si accontenta di dimostrare che la probabilità marginale (i.e. che il generico elemento  $a_i$  della s.sequenza  $a_1, \dots, a_k$  sia nel campione) sia  $s/k$ , ma la vera correttezza di RSA è data dal seguente

**Theorem 3.1.** Per ogni fissato  $s \geq 1$ , ad ogni round  $k \geq s$ , per ogni sottoinsieme  $S' \subseteq \{a_1, \dots, a_k\}$  tale che  $|S'| = s$ , la probabilità che RSA restituisca, al round  $k$ , esattamente  $S'$  è:

$$\mathbb{P}(\mathcal{S} = S') = \frac{1}{\binom{k}{s}} = \frac{s!(k-s)!}{k!}.$$

#### 3.1.1 Prova del Teorema 3.1

La dimostrazione è per induzione su  $k$  dove il caso base è  $k = s$ .

- *Base Induzione* ( $k = s$ ). Sino a che  $k \leq s$ , RSA inserisce in  $\mathcal{S}(s, k)$  i primi  $k$  elementi in input. C'è solo un possibile sottoinsieme di size  $k$ , ovvero  $\langle a_1, \dots, a_k \rangle$  stesso. La probabilità che  $\mathcal{S}(s, K)$  coincida con questo sottoinsieme è  $1/\binom{k}{k} = 1$ . Il caso base è verificato.

- *Passo Induttivo*. Possiamo assumere, per ipotesi induttiva che, dopo aver processato  $k$  elementi, ogni sottoinsieme di dimensione  $s$  degli elementi  $\{a_1, \dots, a_k\}$  abbia probabilità  $1/\binom{k}{s}$  di essere scelto come output  $\mathcal{S}(s, k)$ . Ora consideriamo l'elemento  $a_{k+1}$ . RSA genera un indice casuale  $j \in_u [1, k+1]$  ed abbiamo due eventi disgiunti ed esaustivi che determinano il nuovo sampling  $\mathcal{S}$ : a). Se  $j > s$ , l'elemento  $a_{k+1}$  viene scartato. b). Se  $j \leq s$ , l'elemento  $a_{k+1}$  sostituisce l'elemento in posizione  $j$  in  $\mathcal{S}(s, k+1)$ .

Dobbiamo calcolare la probabilità che un generico sottoinsieme  $S' \subseteq \{a_1, \dots, a_{k+1}\}$  di taglia  $s$  sia il nuovo sample  $\mathcal{S}(s, k+1)$ . Distinguiamo l'analisi in due casi distinti.

**Caso (a):**  $a_{k+1} \notin S'$ . In questo caso, per avere  $\mathcal{S}(s, k+1) = S'$ ,  $S'$  deve essere stato scelto tra i primi  $k$  elementi: questo può avvenire *se e soltanto se* abbiamo la congiunzione di due eventi distinti ed indipendenti: **a.1).** l'elemento  $a_{k+1}$  deve essere stato scartato. L'elemento  $a_{k+1}$  viene scartato se l'indice  $j$  estratto è  $> s$ : La probabilità è  $\frac{(k+1)-s}{k+1}$ ; AND **a.2).**  $S'$  e' stato scelto *nel round precedente* come  $\mathcal{S}(s, k)$ : e questo sottoevento ha probabilità  $\frac{1}{\binom{k}{s}}$  per ipotesi induttiva. Quindi,

$$\mathbb{P}(\mathcal{S}(s, k) = S') = \frac{k+1-s}{k+1} \cdot \frac{1}{\binom{k}{s}} = \frac{k+1-s}{k+1} \cdot \frac{s!(k-s)!}{k!} = \frac{s!(k+1-s)!}{(k+1)!} = \frac{1}{\binom{k+1}{s}}.$$

**Caso (b):**  $a_{k+1} \in S'$ . In questo caso, il sample precedente  $\mathcal{S}(s, k)$  doveva contenere i restanti  $s-1$  elementi di  $S'$  (che sono *fissi!*), più un altro elemento  $x$  (*qualsiasi!*) che è rimpiazzato da  $a_{k+1}$  al round  $k+1$ . Definiamo  $S_{fixed}$  come l'insieme degli altri  $s-1$  elementi che compongono  $S'$ :

$$S_{fixed} = S' - \{a_{k+1}\}$$

Il vecchio sample  $\mathcal{S}(s, k)$ , doveva quindi essere composto così:  $S_k = S_{fixed} \cup \{X\}$ , dove  $X$  è un elemento generico dell'insieme  $\{a_1, \dots, a_k\}$  che *non* può appartenere a  $S_{fixed}$ . Dobbiamo quindi calcolare il numero di configurazioni favorevoli a questo evento. In particolare, in quanti modi possiamo scegliere l'elemento  $X$  che occupava il "posto rimosso" in  $\mathcal{S}(s, k)$ ? Dato che abbiamo  $k$  elementi totali al round  $k$  e ne abbiamo fissati  $s-1$  in  $S_{fixed}$ , il numero di scelte favorevoli per  $X$  è:

$$\text{n. di scelte per } X = k - (s-1) = k - s + 1$$

Esistono quindi esattamente  $(k-s+1)$  diversi sottoinsiemi  $\mathcal{S}(s, k)$  che possono trasformarsi in  $S'$  tramite la sostituzione di  $X$  con  $a_{k+1}$ .

A questo punto possiamo calcolare la probabilità congiunta del caso  $a_{k+1} \in S'$ . Infatti, poiché queste configurazioni sono mutuamente esclusive, la probabilità totale è la somma delle probabilità di ogni singola configurazione:

$$\mathbb{P}(\mathcal{S}(s, k+1) = S') = \sum_{x \in \{a_1, \dots, a_k\} - S_{fixed}} \mathbb{P}((\mathcal{S}(s, k) = S_k \equiv S_{fixed} \cup \{X = x\}) \cdot \mathbb{P}(a_{k+1} \text{ rimpiazza } x))$$

Inoltre osserviamo:

- Per ipotesi induttiva, per ogni  $S_k$  abbiamo  $\mathbb{P}(\mathcal{S}(s, k) = S_k) = 1/\binom{k}{s}$ .
- La probabilità che  $a_{k+1}$  venga inserito è  $\frac{s}{k+1}$ ;
- La probabilità che venga rimpiazzato proprio l'elemento  $x$  (e non uno degli altri  $s-1$  elementi in  $S_{fixed}$ ) è  $\frac{1}{s}$ .

Quindi,

$$\mathbb{P}(\mathcal{S}(s, k+1) = S') = (k-s+1) \cdot \left[ \frac{1}{\binom{k}{s}} \cdot \frac{s}{k+1} \cdot \frac{1}{s} \right]$$

Adesso, svolgiamo alcuni calcoli combinatorici. Sviluppiamo il coefficiente binomiale:

$$\begin{aligned} \mathbb{P}(\mathcal{S}(s, k+1) = S') &= (k-s+1) \cdot \frac{s!(k-s)!}{k!} \cdot \frac{1}{k+1} \\ &= \frac{s!(k-s+1)(k-s)!}{(k+1)k!} \\ &= \frac{s!(k-s+1)!}{(k+1)!} = \frac{1}{\binom{k+1}{s}}. \end{aligned}$$

Questo completa la dimostrazione.

**Remark 3.2.** *Se cerchi una fonte accademica rigorosa, il riferimento standard è: \* \*\*Jeffrey S. Vitter (1985)\*\*\*, \*\*"Random Sampling with a Reservoir"\*\*, ACM Transactions on Mathematical Software. È il paper definitivo che analizza matematicamente RSA e le sue varianti più efficienti.*

### 3.2 Sampling Uniformi ed Ordinati Random da Streaming

Nella Subsection 2.1 abbiamo visto come generare un ordinamento random di un vettore  $A[1 : n]$  mediante l'algoritmo FYA mentre in Subsection 3.1 abbiamo presentato l'algoritmo RSA che genera e mantiene un sample di size fissata da uno streaming di elementi  $a_1, \dots, a_k, \dots$

In questa subsection vogliamo combinare i due algoritmi in modo da mantenere in modo efficiente ed "online" un sampling che abbia entrambe le proprietà suddette, i.e., vogliamo un algoritmo online che risolva il seguente

**Problema P3.** *Su Streaming Input 1, generare e mantenere, ad ogni round  $k \geq 1$ , un sampling ordinato  $\vec{\mathcal{S}}(s, k)$ , di size  $s$ , tale che:*

- a) *sia scelto u.a.r. tra tutti i possibili s.insiemi di size  $s$  di  $\{a_1, \dots, a_k\}$ ;*
- b) *i suoi elementi sono ordinati u.a.r.*

Consideriamo quindi la procedura quì sotto, dove, in accordo con le notazioni utilizzate,  $\vec{\mathcal{S}}(s, k)$  denota l'array *ordinato* che mantiene gli elementi del sample  $\mathcal{S}(s, k)$ . Al fine di semplificare la descrizione della procedura, consideriamo che, ad ogni round  $k$ , esista una copia distinta del vettore  $\vec{\mathcal{S}}(s, k)$  e che si possano fare operazioni su queste diverse copie: Tuttavia, è molto facile verificare che tutte le operazioni richieste ed il mantenimento del sample si possono eseguire usando un solo array e quindi utilizzando spazio e tempo  $\tilde{O}(s)$ .

**Permuting Reservoir Sampling Algorithm** PRSA

**Input:** Input Stream (1);  $s \in [n]$

**Set**  $\vec{\mathcal{S}}(s, 0)[1 : s] := \vec{0}$ ;

**For Each**  $1 \leq k \leq s$  **Do:**

**Set**  $\vec{\mathcal{S}}(s, k)[k] := a_k$ ;

**Choose**  $i \in_u [k]$ ;

**Swap**  $\vec{\mathcal{S}}(s, k)[i]$  with  $\vec{\mathcal{S}}(s, k)[k]$  . \*\*\* Apply one iteration of FYA \*\*\*

**For**  $k = s+1, \dots$ , **Do** \*\*\* Apply the two cases of RSA \*\*\*

**Choose**  $j \in_u [k]$   
**If**  $j \leq s$  **Then:**    \*\* Case (i) \*\*  
**Set**  $\vec{\mathcal{S}}(s, k)[1 : s]$  as follows:  
- **Remove**  $a_j$  from  $\vec{\mathcal{S}}(s, k-1)[1 : s]$ ;  
- The first  $s-1$  elements of  $\vec{\mathcal{S}}(s, k)[1 : s]$  are assigned in the same order as in  $\vec{\mathcal{S}}(s, k-1)[1 : s]$ ;  
- **Set**  $\vec{\mathcal{S}}(s, k)[s] := a_k$   
- **Choose**  $h \in_u [k]$   
- **Swap**  $\vec{\mathcal{S}}(s, k)[s]$  with  $\vec{\mathcal{S}}(s, k)[h]$  \*\* Apply one iteration of FYA \*\*  
**If**  $j \geq s+1$  **Then:**  
- **Discard**  $a_k$ ;    \*\* Case (ii) \*\*  
- **Set**  $\vec{\mathcal{S}}(s, k)[1 : s] := \vec{\mathcal{S}}(s, k-1)[1 : s]$ .  
**Maintain and Return**  $\mathcal{S}(s, k)[1 : s]$ .

**Theorem 3.3.** *L'algoritmo PRSA risolve il problema P3 in spazio e tempo (per round)  $\tilde{O}(s)$ .*

La prova completa è lasciata come esercizio per gli studenti. Qui sotto diamo un breve sketch dei passi più importanti.

*Sketch of Proof.* La dimostrazione del fatto che, ad ogni round  $k$ , il vettore  $\vec{\mathcal{S}}(s, k)[1 : s]$  soddisfa le Proprietà (a) e (b) del problema P3, è ottenuta applicando il Teorema 3.1 e il Lemma 2.2.

In particolare, grazie alla strategia RSA, l'insieme  $\mathcal{S}(s, k)$  è uniforme su  $\binom{k}{s}$ .

Per quanto riguarda l'ordinamento mantenuto nel vettore  $\mathcal{S}(s, k)[1 : s]$  ad ogni round, consideriamo il primo caso (i.e. l'elemento  $a_k$  viene scelto e viene tolto un elemento random dal vecchio sample) e facciamo le seguenti osservazioni.

Mediante una facile induzione su  $k \geq 1$ , possiamo affermare che  $\vec{\mathcal{S}}(s, k-1)[1 : s]$  è ordinato in modo u.a.r.. Ora, al passo  $k$ , condizionando al Caso (i), ed applicando l'ipotesi induttiva per  $k-1$ , il s.vettore  $\vec{\mathcal{S}}(s, k)[1 : s-1]$  è, per costruzione, una *sottopermutazione* u.a.r. degli  $s-1$  elementi che sono sopravvissuti. Allora, se in questa sottopermutazione, inseriamo l'elemento  $a_k$  in una posizione u.a.r.  $h \in_u [s]$ , l'analisi dell'Algoritmo FYA, fatta in Section 2, mostra facilmente che anche il vettore  $\vec{\mathcal{S}}(s, k)[1 : s]$ , alla fine del round  $k$ , è ordinato u.a.r.

Il secondo caso è banale. □

## 4 Merging di più Sampling in un Sistema Distribuito

Come annunciato in Section 1, consideriamo adesso un sistema distribuito a stella formato da un server centrale  $C$  e da  $h \geq 2$  nodi periferici  $P_1, \dots, P_h$ . Quindi, il grafo di comunicazione del sistema distribuito che consideriamo è il seguente:

$$G = (V, E) \text{ dove } V = \{C, P_1, \dots, P_h\} \text{ e } E = \{\{C, P_i\} : i = 1, \dots, h\}$$

Vogliamo risolvere in modo efficiente il seguente

**Problema P4.** *Il nodo centrale  $C$  deve calcolare e mantenere un sampling u.a.r.  $\mathcal{S}$  del data set  $A$  globale che è partizionato in  $m$  sottoinsiemi disgiunti  $A_1, \dots, A_m$ , ognuno memorizzato e gestito privatamente da un nodo periferico. Il nodo centrale non ha accesso diretto ai dati dei nodi periferici.*

L'idea base del protocollo consiste nel generare localmente dei random samplings  $\mathcal{S}_1, \dots, \mathcal{S}_m$  da parte dei singoli nodi periferici e fare in modo che  $C$  possa generare  $\mathcal{S}$  chiedendo gli elementi random dai nodi periferici in modo efficiente. In particolare, il protocollo calcolerà, mediante un processo

aleatorio locale, il numero (random) di elementi  $s_i$  che  $C$  dovrà chiedere al nodo periferico  $P_i$ , per ogni  $i = 1, \dots, h$ , per formare il sample del data set globale  $A$ . Lo scopo, oltre alla correttezza del protocollo, è quello di ottimizzare la message complexity.

Il processo aleatorio che verrà utilizzato è quello generato da una variabile aleatoria *con distribuzione ipergeometrica*. Le proprietà fondamentali della distribuzione ipergeometrica sono descritte nella seguente subsection preliminare.

**Nota.** Considereremo tale distribuzione nel caso in cui l'evento *successo* è binario e questo corrisponderà al modello distribuito in cui vi sono solo due nodi periferici, i.e.,  $h = 2$ . Il caso  $h \geq 3$  necessita di calcoli più complessi che non fanno parte dei nostri obiettivi.

#### 4.1 Preliminaries: La Distribuzione Ipergeometrica

La distribuzione *Ipergeometrica* è il modello matematico fondamentale per il campionamento *senza reinserimento*.

**Definition 4.1** (Distribuzione Ipergeometrica di parametri  $(n, m, s)$ ). *Sia  $A$  un data set di  $n$  elementi distinti: in particolare,  $m$  elementi di successo (o di tipo 1) e  $n - m$  elementi di insuccesso (o di tipo 2). Si estraggono  $s \leq n$  elementi u.a.r, senza reinserimento: questo sottoinsieme random di  $A$  costituisce il Sampling. La variabile casuale  $X$ , che rappresenta il numero di successi nel Sampling, segue una Distribuzione Ipergeometrica, indicata con  $X \sim \text{Hypergeometric}(n, m, s)$ .*

*La sua Probability Mass Function (PMF) è definita per  $k \in \{\max(0, s - (n - m)), \dots, \min(s, m)\}$  come:*

$$P(X = k) = \frac{\binom{m}{k} \binom{n-m}{s-k}}{\binom{n}{s}}$$

**Fact 4.2.** *Il valor medio di  $X \sim \text{Hypergeometric}(n, m, s)$  è  $\mathbb{E}[X] = s \cdot \frac{m}{n}$ . La varianza è  $\text{Var}(X) = s \cdot \frac{m}{n} \cdot \frac{n-m}{n} \cdot \frac{n-s}{n-1}$ .*

Il termine  $\frac{n-s}{n-1}$  nella formula della Varianza è noto come *fattore di correzione per popolazioni finite*, ed è ciò che distingue l'ipergeometrica dalla binomiale (dove le estrazioni sarebbero indipendenti).

**Interpretazione come Processo Aleatorio.** L'interpretazione più naturale della distribuzione ipergeometrica è quella di un processo di estrazione sequenziale da un'urna in cui la probabilità di successo cambia a ogni passo, condizionata dai risultati precedenti.

*Il Processo dell'Urna.* Immaginiamo una sequenza di variabili aleatorie binarie  $Y_1, Y_2, \dots, Y_s$ , dove  $Y_i = 1$  se l' $i$ -esimo elemento estratto è un successo, altrimenti  $Y_i = 0$ . E' facile vedere che in questo processo:

- i) La probabilità del primo successo è  $\mathbb{P}(Y_1 = 1) = \frac{m}{n}$ ;
- ii) La probabilità del secondo successo dipende dal primo:

$$\mathbb{P}(Y_2 = 1 | Y_1 = 1) = \frac{m-1}{n-1} \quad \text{and} \quad \mathbb{P}(Y_2 = 1 | Y_1 = 0) = \frac{m}{n-1};$$

- iii) e così via .....



**Fact 4.3** (Exchangeability). *la sequenza  $Y_1, \dots, Y_n$  è scambiabile. Ciò significa che la probabilità di una specifica sequenza di successi e insuccessi dipende solo dal numero totale di successi  $k$ , e non dall'ordine in cui sono apparsi. Formalmente, per ogni sequenza binaria fissata  $(y_1, \dots, y_s)$  tale che  $\sum y_i = k$ , vale che*

$$P(Y_1 = y_1 \wedge \dots \wedge Y_s = y_s) = \frac{m!(n-m)!(n-s)!}{n!(m-k)!(n-m-(s-k))!}$$

## 4.2 Weighted Hypergeometric Merging of Samplers

L'invarianza per permutazione enunciata nel Fact 4.3 sarà la proprietà chiave che sfrutteremo per fare un merging controllato dei due sample provenienti dai due nodi periferici. Ricordiamo infatti che ci studieremo solo il caso in cui vi sono solo due nodi periferici e quindi il data set iniziale  $U$  di  $n$  elementi è partizionato in due sottoinsiemi  $A$  e  $B$  rispettivamente di  $m$  ed  $n - m$  elementi. Quindi:

- $n = m + \ell$  size data set totale  $U$ ;
- $m =$  size del data set  $A$  del nodo  $P_1$ ;
- $n - m = \ell$  size del data set  $B$  del nodo  $P_1$ ;
- $s =$  size del Sample

I due nodi periferici, utilizzando gli algoritmi descritti nelle section precedenti, possono generare privatamente, rispettivamente, due sample:  $P_1$  genera  $\mathcal{S}_A$  dal data set  $A$  e  $P_2$  genera il sampling  $\mathcal{S}_B$  dal data set  $B$ . Il problema tecnico che dobbiamo affrontare consiste nel definire un protocollo distribuito che permette al nodo centrale  $C$  di ottenere un sampling  $\mathcal{S}$  uniforme su tutto  $A \cup B$  di size  $s$ . La soluzione procede in questo modo.

### Protocollo Weighted Hypergeometric Sampling WHS

**Local Inputs:** parametro  $s$  per  $C$ ; Data Set  $A$  per  $P_1$  and  $B$  per  $P_2$ ;

Il nodo  $C$  richiede i valori  $|A|$  e  $|B|$  rispettivamente a  $P_1$  e  $P_2$ .

$P_1$  invia  $|A|$  e  $P_2$  invia  $|B|$ ;

$C$  setta i parametri  $m := |A|$ ,  $n := m + |B|$ , e genera la variabile aleatoria  $k \sim \text{Hypergeometric}(n, m, s)$ ;

$C$  invia  $k$  a  $P_1$  e  $s - k$  a  $P_2$ ;

$P_1$  estrae<sup>1</sup> un s.insieme  $\mathcal{R}_A$  u.a.r. (senza reinserimento) da  $\mathcal{S}_A$  di size esattamente  $k$  e lo invia a  $C$ ;  $P_2$

estrae un s.insieme  $\mathcal{R}_B$  u.a.r. (senza reinserimento) da  $\mathcal{S}_B$  di size esattamente  $s - k$  e lo invia a  $C$ ;

$C$  memorizza e mantiene  $\mathcal{S} := \mathcal{R}_A \cup \mathcal{R}_B$ .

Mostriamo ora che il protocollo WHS sta sostanzialmente simulando il processo di sampling che accadrebbe se i due flussi fossero mescolati in un'unica urna prima del campionamento.

**Theorem 4.4.** *Assumiamo il sistema distribuito a stella  $G(V, E)$  descritto sopra, e fissiamo: il data set  $U = A \cup B$ , con  $|A| = n$ ,  $|B| = n - m$ , e la size del Random Sample  $s = |\mathcal{S}|$ . Allora, il protocollo WHS genera un random sample  $\mathcal{S}$  u.a.r. di size  $s$ .*

<sup>1</sup>Si noti che questa estrazione può essere effettuata dai nodi  $P_1$  e  $P_2$  localmente sul loro rispettivo set di dati con gli stessi algoritmi che abbiamo visto nelle sezioni precedenti: Per il *Principle of Deferred Decision*, estrarre dal random sampling o direttamente dal data set è la stessa cosa da un punto di vista della distribuzione del sample da inviare a  $C$  di size  $k$  o  $n - k$ .

### 4.3 Prova del Theorem 4.4

---

#### SONO QUI

---

Algoritmo di Merge Distribuito (Hypergeometric Sampling)

Siano  $R_1$  e  $R_2$  due reservoir di dimensione  $s$ , estratti rispettivamente da popolazioni  $D_1$  e  $D_2$  di taglia  $n_1$  e  $n_2$ . L'obiettivo è costruire un nuovo reservoir  $R$  di dimensione  $s$  che sia un \*\*Simple Random Sample (SRS)\*\* dell'unione  $D = D_1 \cup D_2$ , dove  $n = n_1 + n_2$ .

1. Definizione dell'Algoritmo

L'algoritmo si articola nei seguenti passaggi:

1. \*\*Campionamento del numero di successi:\*\* Si genera una variabile casuale  $K$  che segue una distribuzione ipergeometrica:

$$K \sim \text{Hypergeometric}(n, n_1, s)$$

La cui funzione di massa di probabilità è:

$$P(K = k) = \frac{\binom{n_1}{k} \binom{n_2}{s-k}}{\binom{n}{s}}$$

2. \*\*Selezione dai Sotto-campioni:\*\* Si estraggono casualmente  $k$  elementi dal reservoir  $R_1$  senza reinserimento. Si estraggono casualmente  $s-k$  elementi dal reservoir  $R_2$  senza reinserimento.

3. \*\*Composizione:\*\* Il nuovo reservoir  $R$  è l'unione dei due sottoinsiemi estratti.

---

2. Dimostrazione di Uniformità (SRS)

Affinché  $R$  sia un SRS di  $D$ , ogni possibile sottoinsieme  $X \subset D$  di dimensione  $s$  deve avere la stessa probabilità di essere scelto:

$$P(R = X) = \frac{1}{\binom{n}{s}}$$

Analisi della Probabilità Composta

Sia  $X$  un sottoinsieme arbitrario di dimensione  $s$ . Supponiamo che  $X$  contenga  $k$  elementi provenienti da  $D_1$  e  $s-k$  elementi provenienti da  $D_2$ . La probabilità che l'algoritmo produca esattamente  $X$  è data dal prodotto di tre eventi indipendenti:

1. \*\*Evento  $E_1$ :\*\* La variabile ipergeometrica  $K$  assume il valore  $k$ . 2. \*\*Evento  $E_2$ :\*\* Il reservoir  $R_1$  contiene esattamente i  $k$  elementi di  $X \cap D_1$  questi vengono selezionati durante il merge. 3. \*\*Evento  $E_3$ :\*\* Il reservoir  $R_2$  contiene esattamente i  $s-k$  elementi di  $X \cap D_2$  questi vengono selezionati durante il merge.

Calcolo dei Termini

La probabilità che un set specifico di  $k$  elementi sia contenuto in un reservoir di dimensione  $s$  estratto da  $n_1$  è:

$$P(X \cap D_1 \subset R_1) = \frac{\binom{n_1-k}{s-k}}{\binom{n_1}{s}}$$

Una volta che questi elementi sono nel reservoir, la probabilità di sceglierli estraendo  $k$  elementi dai  $s$  disponibili è  $1/\binom{s}{k}$ . Pertanto:

$$P(E_2) = \frac{\binom{n_1-k}{s-k}}{\binom{n_1}{s}} \cdot \frac{1}{\binom{s}{k}}$$

Analogamente per  $D_2$ :

$$P(E_3) = \frac{\binom{n_2-(s-k)}{s-(s-k)}}{\binom{n_2}{s}} \cdot \frac{1}{\binom{s}{s-k}} = \frac{\binom{n_2-s+k}{0}}{\binom{n_2}{s}} \cdot \frac{1}{\binom{s}{s-k}}$$

Prodotto Finale

Sostituendo i termini nella formula della probabilità totale  $P(R = X) = P(K = k) \cdot P(E_2) \cdot P(E_3)$ :

$$P(R = X) = \left[ \frac{\binom{n_1}{k} \binom{n_2}{s-k}}{\binom{n}{s}} \right] \cdot \left[ \frac{\binom{n_1-k}{s-k}}{\binom{n_1}{s} \binom{s}{k}} \right] \cdot \left[ \frac{1}{\binom{n_2}{s} \binom{s}{s-k}} \right]$$

Utilizzando l'identità delle combinazioni  $\binom{n}{k} \binom{k}{r} = \binom{n}{r} \binom{n-r}{k-r}$ , si può dimostrare che i termini relativi a  $n_1$  e  $n_2$  si semplificano, lasciando esclusivamente:

$$P(R = X) = \frac{1}{\binom{n}{s}}$$

Ciò dimostra che il set  $R$  è un campione casuale semplice dell'unione dei flussi.

—

## 5 ALTRO ARGOMENTO PER ADRC: CONSISTENT HASHING

VEDI LESSON 1 IN

<https://web.stanford.edu/class/cs168/>